

Existing File Changes — Math Visualization Tool Integration

Scope: This document covers only modifications to files that already exist in the repository. No new modules are discussed here. Every section names the exact file, what currently exists in it, what needs to be added or changed, and precisely why — i.e., which new capability this change enables and which new module will consume it.

1. `dsl/ast.rs`

What currently exists

`MathExpression` is a recursive enum covering `Variable`, `Constant`, `Number`, `BinaryOp`, `UnaryOp`, `FunctionCall`, `Derivative`, `Integral`, `Limit`, `Summation`, `Product`, `Piecewise`, `MatrixExpr`, and `VectorExpr`. Every variant carries its mathematical semantics but carries **no identity** — there is no way to refer to a specific node from outside the tree, no way to tag a node with meaning beyond its math type, and no way to associate a node with a rendered object in the scene.

`AstFile` currently has fields: `scene`, `library_imports`, `materials`, `fields`, `entities`, `constraints`, `motions`, `math_objects`, `compound_motions`, `trajectories`, `timelines`, and `span`. It carries no concept of educational context.

`AstValue` currently has variants: `Number`, `String`, `Boolean`, `Identifier`, `List`, `Vector`, `Matrix`, `MathExpression`.

Changes required

1.1 Add a `node_id` field to every `MathExpression` variant

What to change: Wrap every `MathExpression` variant into a struct that carries a `node_id: String` alongside the existing data, or add a wrapper type `AnnotatedExpr { node_id: String, expr: MathExpression }` that replaces all recursive `Box<MathExpression>` usages.

The recommended approach is the wrapper type because it does not require modifying the match arms for every variant. Change all recursive children — e.g., `BinaryOp(Box<MathExpression>, MathBinaryOperator, Box<MathExpression>)` — to `BinaryOp(Box<AnnotatedExpr>, MathBinaryOperator, Box<AnnotatedExpr>)`. At parse time, every parsed node is assigned a deterministic node ID derived from its position in the source (line + column + depth index formatted as a string).

Why: The `StepEmitter` (in the new `step_trace` module) emits events keyed by `node_id`. Without stable node IDs on every AST node, there is no way to tell the frontend "highlight the left operand of the multiplication at step 3." The LLM Translator also needs node IDs to produce `highlight_schedule` DSL blocks that reference specific sub-expressions.

What new module consumes this: `step_trace/`, `llm_dsl_translator/`, the `LatexSerializer` (new file within `dsl/`).

1.2 Add `highlight_token` field to `AnnotatedExpr`

What to change: Add `highlight_token: Option<String>` to the `AnnotatedExpr` wrapper. This is a short alphanumeric string (e.g., `"hk_03"`) assigned by the `LLMOrchestrator` to mark which expression nodes are pedagogically significant in the current lesson step. It is `None` for most nodes and `Some(token)` for nodes the LLM has designated as the active teaching focus.

Why: The `HighlightCoordinator` in the `explanation_bus` module reads this token from the AST (via the snapshot) and assigns it a consistent color index. The same token appears in the rendered KaTeX equation in the frontend, in the prose narrative, and in the 3D scene object annotations. Without this field on the AST node, there is no common identifier to tie all three panels together.

What new module consumes this: `explanation_bus/`, `step_trace/`.

1.3 Add `ConceptAnnotation` optional field to `AstFile`

What to change: Add an optional field `concept_ref: Option<ConceptAnnotation>` to `AstFile`. Define `ConceptAnnotation` as a struct with three string fields: `concept_id`, `section_id`, and `step_index: usize`. This is parsed from the new optional `concept_ref { }` block in the DSL grammar (grammar change described below).

Why: The DSL Translator, when generating a scene for a specific lesson step, must embed which concept step this scene illustrates. The `AnimationSyncController` in the `explanation_bus` module reads this annotation from the compiled IR to map concept navigation events (user clicks "next step") to the correct DSL scene file. Without it, the system has no way to know which scene file corresponds to step 3 of the Chain Rule.

What new module consumes this: `llm_dsl_translator/`, `explanation_bus/`.

1.4 Add `AnnotationNode` struct and include it in `AstFile`

What to change: Define a new struct:

```
struct AnnotationNode {
    label_text: String,      // may contain LaTeX between $...$
    anchor_entity_id: String, // refers to an entity name in the same DSL file
    position_offset: [f64; 3], // world-space offset from anchor
    equation_node_id: Option<String>, // links to a MathExpression node_id
    highlight_token: Option<String>,
    span: SourceSpan,
}
```

Add `annotations: Vec<AnnotationNode>` to `AstFile`.

Why: The LLM instructs the Translator to place labels on geometric objects. For example, "label the x-component of the unit circle vector as $\cos\theta$." These labels are not motion or constraint data — they are display

overlays. The frontend's Panel C reads the compiled annotation data and renders HTML labels positioned over the Three.js canvas using CSS absolute positioning. Without this AST node type, there is nowhere in the DSL data model to store this information.

What new module consumes this: `llm_dsl_translator/`, frontend Panel C.

1.5 Add `HighlightScheduleEntry` and include it in `AstFile`

What to change: Define:

```
struct HighlightScheduleEntry {
    at_time: f64,      // timeline timestamp
    highlight_token: String,
    entity_id: String, // which scene entity highlights
    color_index: u8,    // 0-15, matches HighlightCoordinator table
    span: SourceSpan,
}
```

Add `highlight_schedule: Vec<HighlightScheduleEntry>` to `AstFile`.

Why: The LLM produces a timeline of when each geometric object should be highlighted in synchrony with the equation being explained. For instance, "at t=2.0s, highlight the radius vector (color 3) because that is when we evaluate the outer function." The `AnimationSyncController` reads this schedule to emit `BusEvent::StepAdvanced` at the right moments during animation playback. Without this data in the AST, the 3D animation and the equation walkthrough cannot be synchronized.

What new module consumes this: `llm_dsl_translator/`, `explanation_bus/`, `compiler/runtime/snapshot_builder.rs`.

2. `dsl/grammar.md`

What currently exists

The grammar is versioned at v2.0 and declared **FROZEN**. The mandatory block order is: `scene` → `library_imports` → `materials?` → `fields?` → `entities` → `constraints` → `motions` → `compound_motions?` → `trajectories?` → `timelines`. Design goal 6 states: "LLM-safe but LLM-irrelevant." No concept of educational annotation, label overlays, or highlight scheduling exists.

Changes required

2.1 Unfreeze and bump to v2.1

What to change: Change the version header from v2.0 to v2.1. Update the compatibility section to state: "v2.1 adds optional educational annotation blocks. All v2.0 files remain valid — new blocks are optional."

Remove or soften the "LLM-irrelevant" wording in design goal 6. Replace it with: "LLM-safe: LLM systems interact via the structured AnimationIntent interface, not by writing DSL directly. DSL is generated by the Translator, not the LLM."

Why: The grammar is the source of truth for what the DSL compiler accepts. Any new block added to `ast.rs` and `parser.rs` must be first specified here. The freeze note currently suggests any new block is invalid, which would mislead future maintainers.

2.2 Add `concept_ref` optional block specification

What to change: After the `timelines` section in the EBNF grammar, add:

```
concept_ref? ::= "concept_ref" "{"  
    "concept_id:" STRING  
    "section_id:" STRING  
    "step_index:" INTEGER  
    "}"
```

Note it as optional and specify it must appear after `timelines` if present. Add it to the mandatory order list as a final optional entry.

Why: The parser needs a grammar specification to recognize and parse this block. Without the grammar spec, the lexer will emit an error token when it encounters `concept_ref` in a DSL file.

2.3 Add `annotations` optional block specification

What to change: Add after `concept_ref?`:

```
annotations? ::= "annotations" "{" annotation* "}"  
annotation ::= "annotation" STRING "{" annotation_fields "}"  
annotation_fields ::=  
    "anchor:" IDENTIFIER  
    ("offset:" "[" NUMBER "," NUMBER "," NUMBER "]" )?  
    ("label:" STRING)?  
    ("equation_node_id:" STRING)?  
    ("highlight_token:" STRING)?
```

Why: The parser must know the structure of `annotation` blocks to correctly build `AnnotationNode` AST objects. Without this grammar spec, the parser cannot distinguish annotation field names from entity field names and would fail with an invalid-token error.

2.4 Add `highlight_schedule` optional block specification

What to change: Add:

```
highlight_schedule? ::= "highlight_schedule" "{" schedule_entry* "}"
schedule_entry      ::= "at" NUMBER "{"
    "token:" STRING
    "entity:" IDENTIFIER
    "color_index:" INTEGER
"}
```

Why: Same reason as above — the parser needs the grammar to correctly tokenize and parse highlight schedule blocks.

3. `dsl/lexer.rs`

What currently exists

The lexer recognizes all v2.0 keywords: `scene`, `library_imports`, `materials`, `fields`, `entity`, `constraint`, `motion`, `compound_motion`, `trajectory`, `timeline`, and all component-level keywords. It produces `TokenKind` variants for each recognized keyword.

Changes required

3.1 Add new keyword token variants

What to change: In the `TokenKind` enum (or wherever DSL keywords are defined), add:

- `ConceptRef`
- `Annotations`
- `Annotation`
- `HighlightSchedule`
- `At` (for the `at` keyword in schedule entries, distinct from the `at_time` field)
- `AnchorKw` (for the `anchor:` field keyword, if keywords are lexed separately from identifiers in field position)

In the keyword recognition table (the match or hash map that maps string slices to `TokenKind`), add:

- `"concept_ref"` → `TokenKind::ConceptRef`
- `"annotations"` → `TokenKind::Annotations`
- `"annotation"` → `TokenKind::Annotation`
- `"highlight_schedule"` → `TokenKind::HighlightSchedule`
- `"at"` → `TokenKind::At`

Why: Without these token kinds, the lexer emits `TokenKind::Identifier` for `concept_ref`, which the parser would then reject with a syntax error because it expects a known block keyword.

What flows downstream: `parser.rs` consumes these tokens.

4. `dsl/parser.rs`

What currently exists

`parse_scene_document()` orchestrates parsing in mandatory block order. It calls dedicated parse functions for each block: `parse_scene()`, `parse_library_imports()`, `parse_materials()`, `parse_fields_section()`, `parse_entities()`, `parse_constraints()`, `parse_motions()`, `parse_compound_motions()`, `parse_trajectories()`, `parse_timelines()`. The result is assembled into `AstFile`.

The `MathExpression` parsing occurs in `parse_primary_expression()`, `parse_unary_expression()`, and `parse_binary_expression()`. These functions recursively build `MathExpression` enum values but do not assign any node IDs.

Changes required

4.1 Assign `node_id` during `MathExpression` parsing

What to change: In `parse_primary_expression()`, `parse_unary_expression()`, and `parse_binary_expression()` (and any other function that constructs a `MathExpression` value), wrap the resulting expression in `AnnotatedExpr` before returning. The `node_id` is generated as a string concatenation of the current source span's start line, start column, and a per-parse-session counter. A lightweight counter stored in the `Parser` struct is sufficient.

Change the `Parser` struct to add `node_id_counter: usize`. Increment it each time a new `AnnotatedExpr` is created.

Why: The parser is the only place where `MathExpression` nodes are created, so it is the correct and only place to assign stable node IDs. Assigning them later (e.g., in the validator or lowering) is impossible because the tree structure is already fixed.

4.2 Add `parse_concept_ref()` function

What to change: Add a new private function `parse_concept_ref(&mut self) -> DslResult<ConceptAnnotation>`. It checks for `TokenKind::ConceptRef`, parses the block body expecting `concept_id:`, `section_id:`, and `step_index:` fields, and returns a `ConceptAnnotation` struct.

In `parse_scene_document()`, after parsing timelines, add:

```
rust
```

```
let concept_ref = if self.check(TokenKind::ConceptRef) {
    Some(self.parse_concept_ref())
} else {
    None
};
```

Assign `concept_ref` to `AstFile.concept_ref`.

Why: Without this parse function, the parser hits `TokenKind::ConceptRef` after timelines and expects `TokenKind::Eof`, producing a parse error on every Translator-generated DSL file.

4.3 Add `parse_annotations()` function

What to change: Add `parse_annotations(&mut self) -> DslResult<Vec<AnnotationNode>>`. It consumes `TokenKind::Annotations`, opens a brace, and repeatedly calls `parse_single_annotation()` until the closing brace. `parse_single_annotation()` reads one `annotation NAME { ... }` block.

Add the call in `parse_scene_document()` after `parse_concept_ref()`:

```
rust

let annotations = if self.check(TokenKind::Annotations) {
    self.parse_annotations()
} else {
    Vec::new()
};
```

Why: The frontend Panel C needs `AnnotationNode` data to render text labels over the canvas. This data must pass through the parser to reach the IR and ultimately the snapshot.

4.4 Add `parse_highlight_schedule()` function

What to change: Add `parse_highlight_schedule(&mut self) -> DslResult<Vec<HighlightScheduleEntry>>`. It consumes `TokenKind::HighlightSchedule`, parses `at NUMBER { token: STRING entity: IDENTIFIER color_index: INTEGER }` entries.

Add the call in `parse_scene_document()` after `parse_annotations()`.

Why: Without this, DSL files generated by the Translator containing `highlight_schedule` blocks will fail to parse, breaking the entire pipeline.

5. `dsl/validator/schema.rs`

What currently exists

Schema validation (Pass 2) checks that each recognized block has the correct field names and value types. It defines allowed field sets for `entity`, `constraint`, `motion`, and component types. It rejects unknown field names within blocks.

Changes required

5.1 Add schema rules for `concept_ref` block

What to change: In the field validation tables, add an entry for block type `"concept_ref"` with required fields `concept_id: String`, `section_id: String`, and `step_index: Integer`. Mark all three as required.

Why: Without a schema rule, the schema validator either ignores unknown block types silently or rejects them with a generic error. Adding explicit rules produces actionable error messages if the Translator generates a malformed `concept_ref` block (e.g., missing `section_id`).

5.2 Add schema rules for `annotation` entries

What to change: Add a schema entry for block type `"annotation"` with: required field `anchor: Identifier`, optional fields `offset: Vector(3)`, `label: String`, `equation_node_id: String`, `highlight_token: String`.

Why: Same as above — the `TranslationValidator` runs the full DSL compiler including schema validation on every Translator output. Clear schema rules enable the feedback loop to report precise errors back to the `LLMOrchestrator`.

5.3 Add schema rules for `highlight_schedule` entries

What to change: Add schema entry for block type `"highlight_schedule_entry"` (or the internal type name used for `at` entries) with required fields `token: String`, `entity: Identifier`, `color_index: Integer`, and implicitly the `at: Number` from the outer `at NUMBER { }` syntax.

6. `dsl/validator/references.rs`

What currently exists

Pass 3 validates that every identifier reference in the file (e.g., motion target names, constraint object references) refers to an entity that was defined earlier in the file. It maintains a set of defined entity IDs and checks references against it.

Changes required

6.1 Validate `annotation` anchor references

What to change: In the reference resolution pass, after processing entities, add a loop over `ast.annotations`. For each `AnnotationNode`, check that `anchor_entity_id` exists in the defined entity set. If not, emit a reference error.

Why: The Translator may generate annotation anchors that reference entity names. If an entity name is misspelled or if the entity definition was not generated correctly, this validator catches it before the IR lowering stage — which would panic or produce invalid output without this check.

6.2 Validate `highlight_schedule` entity references

What to change: For each `HighlightScheduleEntry`, check that `entity_id` exists in the defined entity set. Emit a reference error if not.

Why: Same reason — the schedule references entities by name. A Translator bug could produce a schedule entry referencing a non-existent entity. Catching this in the validator (rather than at runtime) produces a clean error message returned to the LLMOrchestrator.

6.3 Validate `annotation.equation_node_id` references

What to change: Collect all `node_id` values from all `AnnotatedExpr` nodes in `ast.math_objects` (and in motion expression fields). For each annotation with a non-None `equation_node_id`, verify that ID appears in the collected set. Emit an error if the node ID does not exist.

Why: The LLM may hallucinate a node ID that does not correspond to any real AST node. This check prevents the frontend from attempting to highlight a non-existent equation term.

7. `dsl/lower_to_ir.rs`

What currently exists

`IrLowering::lower()` maps each AST node type to a corresponding IR struct. `lower_math_expression()` currently produces `IrMathExpression { expression_type, source, complexity }` — a simplified flat representation that discards the tree structure and only keeps a string serialization and a complexity score.

`IrScene` currently contains: `metadata`, `entities`, `constraints`, `motions`, `math_entities`, `compound_motions`, `timelines`.

Changes required

7.1 Extend `IrMathExpression` to carry `node_id` and `highlight_token`

What to change: Add fields to `IrMathExpression`:

rust

```
pub struct IrMathExpression {
    pub expression_type: String,
    pub source: String,
    pub complexity: usize,
    pub node_id: String,           // NEW
    pub highlight_token: Option<String>, // NEW
    pub children: Vec<IrMathExpression>, // NEW — preserves tree structure
}
```

Change `lower_math_expression()` to recursively lower the `AnnotatedExpr` tree, preserving children. The `children` field contains the lowered left and right operands (for binary ops), or the single inner expression (for unary ops and derivatives), or the argument list (for function calls).

Why: The step trace system needs to traverse the IR expression tree at runtime to emit events node-by-node. Currently the IR expression is a flat string — there is no tree to traverse. Without `children`, the `StepEmitter` cannot walk the expression and emit one event per sub-expression.

Why `highlight_token` here: The IR is what the runtime snapshot builder reads. Having the token on the IR node means the snapshot builder can populate `active_highlight_token` without needing to re-access the original AST (which is not available at runtime).

7.2 Add lowering for `annotations` to `IrScene`

What to change: Add `annotations: Vec<IrAnnotation>` to `IrScene`. Define:

rust

```
pub struct IrAnnotation {
    pub label_text: String,
    pub anchor_entity_id: String,
    pub position_offset: [f64; 3],
    pub equation_node_id: Option<String>,
    pub highlight_token: Option<String>,
}
```

In `IrLowering::lower()`, add a call to `lower_annotations(ast.annotations)` and assign the result to `IrScene.annotations`.

Why: The renderer bridge reads the snapshot which is built from the IR. Annotation data must flow through the IR to reach the renderer snapshot and ultimately the frontend.

7.3 Add lowering for `highlight_schedule` to `IrScene`

What to change: Add `highlight_schedule: Vec<IrHighlightEntry>` to `IrScene`. Define:

```
rust

pub struct IrHighlightEntry {
    pub at_time: f64,
    pub highlight_token: String,
    pub entity_id: String,
    pub color_index: u8,
}
```

Lower `ast.highlight_schedule` into this list in `IrLowering::lower()`.

Why: The `AnimationSyncController` in `explanation_bus` needs the highlight schedule at the IR level so the runtime engine (which has access to `IrScene`) can make it available through the snapshot mechanism.

7.4 Add lowering for `concept_ref` to `IrScene`

What to change: Add `concept_ref: Option<IrConceptRef>` to `IrScene`. Define:

```
rust

pub struct IrConceptRef {
    pub concept_id: String,
    pub section_id: String,
    pub step_index: usize,
}
```

Lower `ast.concept_ref` in `IrLowering::lower()`.

Why: The session manager on the server side reads `IrScene.concept_ref` to route the loaded scene to the correct concept page, enabling the curriculum navigator in the frontend to reflect the current lesson position.

8. `compiler/runtime/src/math/runtime_engine.rs`

What currently exists

`RuntimeMathEngine` has three public methods: `evaluate_expression()`, `evaluate_derivative()`, `evaluate_integral()`. `evaluate_expression()` calls `evaluate_scalar()`, which is a private recursive function that matches on `Expression` variants and computes numerical values. Results are cached in `function_cache`, `derivative_cache`, and `integral_cache`.

The evaluation is entirely silent — it computes a value and returns it. No events are emitted. No intermediate results are recorded.

Changes required

8.1 Add a `step_emitter` field to `RuntimeMathEngine`

What to change: Add a field:

```
rust
pub struct RuntimeMathEngine {
    function_cache: HashMap<String, MathValue>,
    derivative_cache: HashMap<(String, String), Expression>,
    integral_cache: HashMap<(String, String, u64, u64), f64>,
    step_emitter: Option<Arc<Mutex<StepEmitter>>>, // NEW
}
```

`StepEmitter` is defined in the new `step_trace` module. The field is `Option` so the engine works identically without an emitter attached (backward compatible — existing tests need zero changes).

Add a constructor variant `RuntimeMathEngine::with_emitter(emitter: Arc<Mutex<StepEmitter>>) -> Self` alongside the existing `new()`.

Why: The engine must be able to emit step events without coupling to the emitter at construction time (the engine is constructed before the session and emitter are set up). The `Option<Arc<Mutex<...>>>` pattern lets the `RuntimeEngine` inject the emitter after construction via a `set_emitter()` method.

8.2 Instrument `evaluate_scalar()` to emit step events

What to change: In `evaluate_scalar()`, before the match arm computes and returns its result, add a call to emit a step event. The pattern for each arm is:

For `Expression::Binary(lhs, op, rhs)`:

1. Compute `l = evaluate_scalar(lhs, variables)?` — this recursively emits events for the left subtree.
2. Compute `r = evaluate_scalar(rhs, variables)?` — this recursively emits events for the right subtree.
3. Compute `result = l op r`.
4. Call `self.emit_step(node_id_from_expr, op_name, Some(l), Some(r), result, variables)`.
5. Return `Ok(result)`.

For `Expression::FunctionCall(name, args)`:

1. Evaluate all args.

2. Compute result.
3. Call `self.emit_step(node_id, name, None, None, result, variables)`.

For `Expression::Variable(name)`:

1. Look up value.
2. Call `self.emit_step(node_id, "substitute", None, None, value, variables)` with a description like "substitute x = 2.71".

Add a private helper `emit_step()` that constructs a `MathStepEvent` (defined in the new `step_trace` module) and calls `emitter.lock().unwrap().emit(event)` if `self.step_emitter.is_some()`.

Why: This is the core instrumentation. Every sub-expression evaluation produces one event. The frontend's Equation Execution Panel consumes these events one by one when the user presses "Next Step." Without this instrumentation, the runtime is a black box — values appear without any record of how they were computed.

Backward compatibility: The `emit_step()` helper is a no-op when `self.step_emitter.is_none()`. All existing tests pass unchanged.

8.3 Add `node_id` extraction from `Expression`

What to change: Since the `Expression` type in `compiler/runtime/src/math/expression.rs` is distinct from the DSL's `MathExpression` (it is the runtime's own expression type), both need node ID support. Add a `node_id: Option<String>` field to the runtime `Expression` enum's wrapper type (or add it as an optional field on each variant using the same `AnnotatedExpr` pattern).

When the `SceneLoader` loads an `IrScene` and converts `IrMathExpression` objects into runtime `Expression` objects, it must carry the `node_id` through into the runtime `Expression`.

Why: `evaluate_scalar()` needs to access the node ID of each expression node to include it in the emitted step event. Without the node ID on the runtime `Expression`, the emitted event has no way to tell the frontend which specific term in the displayed equation is being computed.

What file to also change: `compiler/runtime/loader.rs` — the conversion from `IrMathExpression` to runtime `Expression` must preserve `node_id`. See section 13 below.

9. `compiler/runtime/executor/step.rs`

What currently exists

`StepKind` is an enum with six variants: `Initialize`, `ApplyConstraints`, `UpdateMotion`, `ValidateState`, `Synchronize`, `Custom`.

`StepMetrics` has five fields: `execution_time_us`, `iterations`, `residual`, `objects_updated`, `constraints_evaluated`.

`ExecutionStep` has two fields: `kind: StepKind` and `description: String`.

Changes required

9.1 Extend `StepKind` with math-level step variants

What to change: Add the following variants to `StepKind`:

```
rust

pub enum StepKind {
    // ... existing variants unchanged ...

    /// Evaluating a specific sub-expression node
    EvaluateSubExpression,
    /// Substituting a variable with its current value
    SubstituteVariable,
    /// Applying a named mathematical rule (chain rule, product rule, etc.)
    ApplyRule,
    /// A simplification was performed
    Simplify,
    /// Convergence check for iterative solving
    ConvergenceCheck,
    /// The active teaching step — wraps any of the above with educational context
    TeachingStep,
}
```

Why: The `ExecutionPipeline` and `StageExecutor` build pipelines from `ExecutionStep` objects. The new `WorkedExample`-driven execution (defined in the new `llm_orchestrator` module) needs to create `ExecutionStep` objects with these kinds so the step history in the Equation Panel shows meaningful labels rather than generic "UpdateMotion" descriptions.

Backward compatibility: All existing code matches on the existing variants. The new variants are only constructed by new code. The `Custom` variant serves as a fallback for anything not covered.

9.2 Extend `StepMetrics` with active-expression fields

What to change: Add three new fields to `StepMetrics`:

```
rust
```

```
pub struct StepMetrics {
    // ... existing fields unchanged ...

    /// Node ID of the MathExpression node active during this step
    pub active_expression_node_id: Option<String>,
    /// Current highlight token active during this step
    pub active_highlight_token: Option<String>,
    /// Human-readable description of the mathematical operation
    pub step_description: Option<String>,
}
```

Update `Default::default()` to initialize all three new fields to `None`.

Why: The snapshot builder reads `StepMetrics` from the most recently executed step and copies `active_highlight_token` into the `RendererSnapshot` (see section 10). This is how the renderer knows which object to highlight on each frame.

9.3 Extend `ExecutionStep` with `equation_node_id` field

What to change: Add `equation_node_id: Option<String>` to `ExecutionStep`. This allows an execution step to be associated with a specific equation sub-expression, enabling the Equation Panel to highlight the corresponding term when this step executes.

Why: When the `WorkedExample` subsystem creates an execution pipeline, each stage corresponds to one mathematical step. Associating each `ExecutionStep` with the node ID of the expression being evaluated is what creates the synchronized highlighting behavior.

10. `compiler/runtime/snapshot_builder.rs`

What currently exists

`RendererSnapshot` has six fields: `tick`, `timestamp`, `objects`, `math_values`, `math_preview`, `math_renderables`, `focus_ids`.

`SnapshotBuilder::build_snapshot()` reads from `RuntimeState` and populates all six fields.

`build_math_renderables()` hard-codes reading specific named parameters (`wave`, `decay`, `time`, `engine.integral_x2_0_1`) from the state and produces `SnapshotMathRenderable` variants. This is a demo-mode function, not a general-purpose one.

Changes required

10.1 Add `active_highlight_token` field to `RendererSnapshot`

What to change: Add `(pub active_highlight_token: Option<String>)` to `(RendererSnapshot)`. This represents the currently active highlight token — the one the frontend should use to highlight a term in the equation and the corresponding 3D object simultaneously.

In `(build_snapshot())`, read the active highlight token from the `(RuntimeState)`. The `(RuntimeState)` gets it from the `(StepEmitter)` (which the `(RuntimeMathEngine)` populates). Add a field `(current_highlight_token: Option<String>)` to `(RuntimeState)` (see section 12) and copy it here.

Why: The renderer bridge reads this from the snapshot in its `(update())` method (see section 11) to call `(apply_focus)` with the right highlight token. Without this field, the bridge has no way to know what to highlight.

10.2 Add `(highlight_schedule)` field to `(RendererSnapshot)`

What to change: Add `(pub highlight_schedule: Vec<SnapshotHighlightEntry>)` to `(RendererSnapshot)`. Define:

```
rust

pub struct SnapshotHighlightEntry {
    pub at_time: f64,
    pub highlight_token: String,
    pub entity_id_hash: u64, // hashed from entity string ID
    pub color_index: u8,
}
```

In `(build_snapshot())`, read `(RuntimeState.highlight_schedule)` (see section 12) and convert each entry to `(SnapshotHighlightEntry)` by hashing the entity ID string to a `(u64)` using the same `(object_id_hash())` method already in the file.

Why: The `(AnimationSyncController)` in the `(explanation_bus)` module needs to know the full highlight schedule for the current scene so it can emit `(BusEvent::StepAdvanced)` at the right timeline moments during auto-advance playback.

10.3 Add `(annotations)` field to `(RendererSnapshot)`

What to change: Add `(pub annotations: Vec<SnapshotAnnotation>)` to `(RendererSnapshot)`. Define:

```
rust
```



```
pub struct SnapshotAnnotation {
    pub label_text: String,
    pub anchor_object_id: u64, // hashed from entity string ID
    pub position_offset: [f64; 3],
    pub equation_node_id: Option<String>,
    pub highlight_token: Option<String>,
    pub is_active: bool, // true if highlight_token matches current active token
}
```

In `build_snapshot()`, populate from `RuntimeState.annotations` (see section 12). Set `is_active = true` when the annotation's `highlight_token` matches `state.current_highlight_token`.

Why: The frontend Panel C renders label overlays by reading annotation data from each snapshot. The `is_active` flag tells the frontend whether to render the label with full opacity or dimmed, and whether to draw an animated connector line to the equation panel.

10.4 Generalize `build_math_renderables()` away from hard-coded parameters

What to change: Replace the current hard-coded reading of `wave`, `decay`, and `engine.integral_x2_0_1` from state parameters with a loop over `state.math_renderables` — a new field added to `RuntimeState` (see section 12). Each entry describes one renderable math object with its type, parameters, and domain. The function maps these entries to `SnapshotMathRenderable` variants.

Why: The current implementation only works for the demo scene. The Translator generates scenes with any mathematical function or surface. The snapshot builder must render whatever the current scene defines, not a hard-coded wave function.

11. `compiler/renderer/bridge.rs`

What currently exists

`RendererBridge::update()` takes a `&RuntimeSnapshot` (called `RuntimeSnapshot` in this file, referring to `RendererSnapshot` from the snapshot builder). It:

1. Checks frame timing via `frame_sync`.
2. Filters visible objects via `visibility.filter()`.
3. Interpolates if enabled.
4. Creates or updates objects in the backend.
5. Generates math objects from renderables.
6. Removes stale objects.

7. Updates stats.
8. Stores the last snapshot.

Highlight behavior comes from `snapshot.focus_ids` — a `Vec<u64>` of numeric object IDs. Highlighting is binary: an object is either highlighted or not.

Changes required

11.1 Read `active_highlight_token` from snapshot and call `apply_focus` with it

What to change: After the step that calls `self.visibility.filter()`, add:

```
rust
// Apply equation-driven highlighting
if let Some(token) = &snapshot.active_highlight_token {
    // Find object IDs registered for this token via highlight schedule
    let token_object_ids = self.resolve_highlight_token(token, &snapshot.highlight_schedule);
    self.visibility.apply_focus_with_token(&mut render_objects, &token_object_ids, token);
}
```

Add a private method `resolve_highlight_token(&self, token: &str, schedule: &[SnapshotHighlightEntry]) -> Vec<u64>` that filters the highlight schedule for entries matching the given token and returns their `entity_id_hash` values.

Why: Currently `apply_focus` is driven by `focus_ids`, which is populated with a `vec![]` TODO comment in `build_snapshot()`. The new mechanism drives focus from the active highlight token, which comes from the step trace system. This is what causes the 3D object to pulse when the corresponding equation term is being explained.

11.2 Pass `color_index` to the backend for colored highlighting

What to change: Extend the call to `backend.set_highlighted(render_id, highlighted)` to `backend.set_highlighted_with_color(render_id, highlighted, color_index: u8)`. The `color_index` comes from the highlight schedule entry for the active token.

This requires adding `set_highlighted_with_color()` to the `RenderBackend` trait (see section 14) and implementing it in the Three.js and Babylon.js backends.

Why: Currently all highlighted objects use the same visual treatment (implied by the backend's default highlight style). The system requires color-coded highlighting — the same term highlighted in orange in the equation must produce an orange glow on the 3D object. This requires the color index to flow from the token → schedule → bridge → backend → JavaScript.

11.3 Emit annotation data to backend

What to change: After processing all objects, add a new block that processes `snapshot.annotations`. For each annotation where `anchor_object_id` maps to an existing `render_id` in `scene_map`, call a new backend method `backend.set_annotation(render_id, label_text, offset, is_active)`. If the object does not exist yet in `scene_map`, skip the annotation for this frame.

Why: Annotations are text labels floating over 3D objects. They must be communicated to the JavaScript rendering backend so Three.js can create CSS2D objects (HTML positioned over the canvas) for each label.

12. `compiler/runtime/state/runtime_state.rs`

What currently exists

`RuntimeState` is a struct with fields `world: WorldState`, `time: TimeState`, and `math_values: HashMap<String, MathValue>`. It has `validate()`, `has_nan()`, `has_infinity()`, and `summary()` methods.

Changes required

12.1 Add `current_highlight_token` field

What to change:

```
rust

pub struct RuntimeState {
    pub world: WorldState,
    pub time: TimeState,
    pub math_values: HashMap<String, MathValue>,
    pub current_highlight_token: Option<String>, // NEW
    pub highlight_schedule: Vec<HighlightScheduleEntry>, // NEW
    pub annotations: Vec<AnnotationEntry>, // NEW
    pub math_renderables: Vec<MathRenderableEntry>, // NEW
}
```

Where `HighlightScheduleEntry` is a copy of `IrHighlightEntry` re-typed for the runtime state layer, `AnnotationEntry` mirrors `IrAnnotation`, and `MathRenderableEntry` describes a mathematical object to be rendered (function, surface, field) with its current parameter values resolved from `world.parameters`.

Update `Default` and `new()` to initialize all new fields to empty/None.

Why: The snapshot builder reads from `RuntimeState`. The new fields give it the data needed to populate the new `RendererSnapshot` fields defined in section 10. These fields are written by the `SceneLoader` when loading a scene (from `IrScene` data) and updated by the `RuntimeEngine` during execution (specifically, `current_highlight_token` is updated each time the `StepEmitter` emits a new step event).

12.2 Add a `set_highlight_token()` method

What to change: Add:

```
rust

pub fn set_highlight_token(&mut self, token: Option<String>) {
    self.current_highlight_token = token;
}
```

Why: The `RuntimeEngine` calls this method in response to step events from the `StepEmitter`. It needs a clean API rather than direct field access.

13. `compiler/runtime/loader.rs`

What currently exists

`SceneLoader::load_scene()` converts an `IrScene` into a `RuntimeState`. It loads entities (converting to `ObjectState`), creates a `ParameterState` with a time parameter and constraint-derived parameters, loads constraints, and returns `RuntimeState::new(world, time)`.

No annotation, highlight schedule, or concept ref data is loaded. No math renderable entries are created. The IR's `math_entities` are also not loaded — they are ignored.

Changes required

13.1 Load `IrScene.annotations` into `RuntimeState.annotations`

What to change: After the existing constraint loading block, add:

```
rust

// Load annotations
let annotations: Vec<AnnotationEntry> = ir.annotations.iter()
    .map(|a| AnnotationEntry {
        label_text: a.label_text.clone(),
        anchor_entity_id: a.anchor_entity_id.clone(),
        position_offset: a.position_offset,
        equation_node_id: a.equation_node_id.clone(),
        highlight_token: a.highlight_token.clone(),
    })
    .collect();
```

Store in `world.annotations` (or as a separate field set on the returned `RuntimeState`).

Why: Annotation data originates in the DSL file, passes through the IR, and must reach `RuntimeState` so the snapshot builder can include it in each frame's snapshot.

13.2 Load `IrScene.highlight_schedule` into `RuntimeState.highlight_schedule`

What to change: Similarly convert `ir.highlight_schedule` entries to `HighlightScheduleEntry` values in `RuntimeState`. The entity ID string must be hashed to a `u64` using `object_id_hash()` (already defined in the file) so it can be matched against numeric object IDs at render time.

Why: Without loading the schedule into runtime state, the snapshot builder (which reads from runtime state, not directly from IR) cannot populate `RendererSnapshot.highlight_schedule`.

13.3 Load `IrScene.math_entities` into `RuntimeState.math_renderables`

What to change: Currently `ir.math_entities` (functions, curves, surfaces, fields) is loaded by the IR lowering but ignored by the scene loader. The loader must convert each `IrMathEntity` into a `MathRenderableEntry` and store it in `RuntimeState.math_renderables`.

`MathRenderableEntry` carries: the entity type (function, surface, field), the expression for the function body in runtime `Expression` form, domain bounds, resolution, and references to parameters by name (so the snapshot builder can look up current values when building `SnapshotMathRenderable` objects).

Converting `IrMathExpression` to runtime `Expression` already partially happens here. This change requires that conversion to also carry `node_id` (see section 8.3).

Why: This replaces the hard-coded `build_math_renderables()` demo logic in `snapshot_builder.rs` with a general-purpose system that works for any mathematical scene the Translator generates.

14. `compiler/runtime/state/parameter_state.rs`

What currently exists

`Parameter` struct has: `id`, `kind`, `value`, `range: Option<ParameterRange>`, `user_controllable`, `derived`, `derivation: Option<String>`, `units: Option<String>`.

`ParameterState` provides `add()`, `remove()`, `get()`, `set()`, `all()`, `user_controllable()`, `derived()`, `values()`, `validate()`.

Changes required

14.1 Add `display_symbol` field to `Parameter`

What to change: Add `(pub display_symbol: Option<String>)` to `Parameter`. This is the LaTeX symbol for the parameter (e.g., `"\theta"` for an angle, `"\omega"` for angular velocity).

Why: The Variable Inspector in the frontend renders each parameter using its LaTeX symbol. Without this field, the inspector can only display the internal string ID (e.g., `"angular_velocity"`), which is not user-facing friendly.

What new module consumes this: Frontend Variable Inspector, `llm_orchestrator` (which sets `display_symbol` when registering parameters for a lesson).

14.2 Add `description` field to `Parameter`

What to change: Add `(pub description: Option<String>)` to `Parameter`. This is a plain-English sentence describing the variable (e.g., "The angle of rotation of the radius vector from the positive x-axis").

Why: The Variable Inspector shows a tooltip when the user hovers over a variable chip. Without this field, there is no description to show.

14.3 Add `animation_entity_id` field to `Parameter`

What to change: Add `(pub animation_entity_id: Option<String>)` to `Parameter`. When not None, this names the scene entity whose geometric representation encodes this variable's value (e.g., the length of a vector arrow represents an angle, or the height of a bar represents a sum).

Why: When the user moves a Variable Inspector slider, the system must know which 3D object to animate in response. The `RuntimeEngine` reads `animation_entity_id` from the changed parameter and triggers a targeted update of that object's geometry rather than recompiling the entire scene.

15. `compiler/renderer/visibility.rs`

What currently exists

`apply_focus(&self, objects: &mut [ObjectState], focus_ids: &[u64])` sets `highlighted = true` on objects whose ID is in `focus_ids` and `highlighted = false` on all others.

The `highlighted` field on `ObjectState` is a plain `bool`.

Changes required

15.1 Add `apply_focus_with_token()` method

What to change: Add a new method:

```
rust
```

```

pub fn apply_focus_with_token(
    &self,
    objects: &mut [ObjectState],
    token_object_ids: &[u64],
    active_token: &str,
) {
    for obj in objects.iter_mut() {
        if token_object_ids.contains(&obj.id) {
            obj.highlighted = true;
            obj.highlight_token = Some(active_token.to_string()); // new field
        } else {
            obj.highlighted = false;
            obj.highlight_token = None;
        }
    }
}

```

Why: The existing `apply_focus` only sets a boolean. The new method also records which token caused the highlight, so the backend can retrieve the associated color index from the highlight schedule and pass it to the shader/material. The token string on the object state is how the bridge knows which color to apply.

15.2 Add `highlight_token: Option<String>` to `ObjectState`

What to change: In `compiler/renderer/mod.rs` (or wherever `ObjectState` for the renderer is defined), add `pub highlight_token: Option<String>` to the renderer's `ObjectState` struct. Initialize to `None` in all existing construction sites (the default value does not break anything).

Why: The bridge reads `obj.highlight_token` when calling `backend.set_highlighted_with_color()`. Without this field, the color index cannot be communicated from the visibility layer to the backend.

16. `compiler/renderer/backend/traits.rs`

What currently exists

`RenderBackend` trait has `set_highlighted(id: u64, highlighted: bool)` as the highlight method. It conveys only a boolean — highlighted or not.

Changes required

16.1 Add `set_highlighted_with_color()` to `RenderBackend` trait

What to change: Add a new required method to the trait:

```
rust
fn set_highlighted_with_color(
    &mut self,
    id: u64,
    highlighted: bool,
    color_index: u8,
) -> RenderResult<()>;
```

Provide a default implementation that falls back to `set_highlighted()` ignoring the color, so all existing backend implementations (Native, Mock) compile without change. Only the Three.js and Babylon.js backends need to actively implement this.

Why: The Three.js and Babylon.js backends must set a specific highlight color (not just on/off) so the 3D object glows with the same color used to highlight the corresponding equation term in the frontend. The color must be communicated to the JavaScript layer.

16.2 Add `set_annotation()` to `RenderBackend` trait

What to change: Add:

```
rust
fn set_annotation(
    &mut self,
    anchor_id: u64,
    label_text: &str,
    offset: [f64; 3],
    is_active: bool,
) -> RenderResult<()>;
```

Provide a no-op default implementation so existing backends compile unchanged.

Why: Three.js backend must create CSS2DObject labels via JS interop. Babylon.js must create GUI labels. Without this method in the trait, the bridge cannot call it polymorphically.

17. `compiler/renderer/backend/three_js.rs`

What currently exists

`ThreeJsBackend` implements `set_highlighted()` with a simple `obj.highlighted = bool` assignment and no color logic. On WASM targets, the backend calls JavaScript via `js_sys` bindings. `create_material_js()` creates a `MeshStandardMaterial` with color, metalness, roughness, and opacity. No emissive color logic exists.

Changes required

17.1 Implement `set_highlighted_with_color()`

What to change: Add the implementation of `set_highlighted_with_color()` for `ThreeJsBackend`. On WASM, call a new JavaScript function `window.setObjectHighlightColor(renderObjectId, highlighted, colorIndex)` via `js_sys::eval` or a dedicated `wasm_bindgen` extern declaration. On non-WASM, log the call.

The JavaScript function `setObjectHighlightColor` must be defined in the frontend JavaScript initialization code (outside the Rust/WASM boundary). It looks up the Three.js mesh by render ID and updates its `material.emissive` color to the color corresponding to `colorIndex` in a 16-entry color table.

Why: The color table must be defined once and shared between the equation panel renderer (which applies CSS color to `<span>` elements) and the Three.js material emissive color. They must use the same color values so the user perceives orange both on the equation term and on the 3D object.

17.2 Implement `set_annotation()`

What to change: Implement `set_annotation()` to call a JavaScript function `window.setObjectAnnotation(anchorId, labelText, offsetX, offsetY, offsetZ, isActive)` on WASM. The JavaScript function creates or updates a `CSS2DObject` containing the label text, positioned at the anchor object's world position plus offset. It shows/hides the label or applies active/inactive CSS classes based on `isActive`.

Why: CSS2D annotations require Three.js `CSS2DRenderer` to be initialized on the JavaScript side. The Rust backend only needs to communicate the label content and position; all rendering machinery is in JavaScript.

18. `compiler/runtime/engine.rs`

What currently exists

`RuntimeEngine` has `math_engine: RuntimeMathEngine` as a field. It calls `math_engine.evaluate_expression()` in `evaluate_runtime_math()`, which is called during initialization and after parameter changes. The result is stored in `state.math_values`. No step emission, no highlight token management.

Changes required

18.1 Add `step_emitter` field to `RuntimeEngine`

What to change: Add `step_emitter: Option<Arc<Mutex<StepEmitter>>>` to `RuntimeEngine`. In `new()`, initialize to `None`. Add a public method `set_step_emitter(emitter: Arc<Mutex<StepEmitter>>)` that stores the emitter and calls `self.math_engine.set_emitter(emitter.clone())` to wire it into the math engine.

Why: The server-side session manager creates the `StepEmitter` after creating the `RuntimeEngine` (because the emitter needs a WebSocket sender reference). The deferred injection pattern allows this.

18.2 Update state's `current_highlight_token` on step emission

What to change: Register a callback (or poll after evaluation) with the `StepEmitter` so that after each call to `evaluate_runtime_math()`, the most recently emitted step event's `highlight_token` is written to `self.state.current_highlight_token`.

Concretely: after `self.math_engine.evaluate_expression()` returns, check if the emitter has a pending last-emitted token, and call `self.state.set_highlight_token(last_token)`.

Why: The snapshot builder reads `state.current_highlight_token`. Without updating it here, the rendered snapshot never reflects the active expression, and the frontend never highlights anything.

18.3 Load annotation and highlight schedule data into state on `initialize()`

What to change: In `RuntimeEngine::initialize()`, after validating the state, call:

```
rust
self.state.highlight_schedule = plan.highlight_schedule.clone();
self.state.annotations = plan.annotations.clone();
```

This requires adding optional `highlight_schedule` and `annotations` fields to `ExecutionPlan` (or passing them separately).

Why: The `ExecutionPlan` is built by the session manager from the compiled `IrScene`. The highlight schedule and annotations from the IR must reach the runtime state so the snapshot builder includes them. The `initialize()` method is the correct injection point because it is called once after loading, before any execution.

Summary Table

File	Nature of Change	Enables
<code>dsl/ast.rs</code>	Add <code>AnnotatedExpr</code> wrapper, <code>node_id</code> , <code>highlight_token</code> , <code>ConceptAnnotation</code> , <code>AnnotationNode</code> , <code>HighlightScheduleEntry</code> fields/types to <code>AstFile</code>	Step trace, annotation display, curriculum routing
<code>dsl/grammar.md</code>	Add <code>concept_ref</code> , <code>annotations</code> , <code>highlight_schedule</code> block specs; bump to v2.1	Parser accepting Translator output
<code>dsl/lexer.rs</code>	Add keyword token variants for new block names	Parser recognizing new blocks without errors

File	Nature of Change	Enables
dsl/parser.rs	Add parse functions for new blocks; assign <code>node_id</code> during <code>MathExpression</code> parsing	AST nodes with stable IDs; new block AST objects
dsl/validator/schema.rs	Add schema rules for new block types	Meaningful validation errors on Translator output
dsl/validator/references.rs	Validate annotation anchors, schedule entity refs, and equation node ID refs	Catching Translator hallucinations early
dsl/lower_to_ir.rs	Extend <code>IrMathExpression</code> with <code>node_id</code> , <code>highlight_token</code> , <code>children</code> ; lower annotations, schedule, concept_ref	Full tree IR; annotation and schedule in IR
compiler/runtime/src/math/runtime_engine.rs	Add <code>step_emitter</code> field; instrument <code>evaluate_scalar()</code> to emit step events; add <code>node_id</code> support	Transparent equation execution, step trace events
compiler/runtime/executor/step.rs	Extend <code>StepKind</code> with math step variants; extend <code>StepMetrics</code> with active expression fields; extend <code>ExecutionStep</code> with <code>equation_node_id</code>	Rich step history for Equation Panel
compiler/runtime/snapshot_builder.rs	Add <code>active_highlight_token</code> , <code>highlight_schedule</code> , <code>annotations</code> to <code>RendererSnapshot</code> ; generalize <code>build_math_renderables()</code>	All panel data flows through each snapshot
compiler/renderer/bridge.rs	Read <code>active_highlight_token</code> and call <code>apply_focus_with_token()</code> ; pass color to backend; emit annotations to backend	Color-coded 3D highlighting; floating labels
compiler/runtime/state/runtime_state.rs	Add <code>current_highlight_token</code> , <code>highlight_schedule</code> , <code>annotations</code> , <code>math_renderables</code> fields	Snapshot builder has all data; state reflects active step
compiler/runtime/loader.rs	Load annotations, highlight schedule, math entities from IR into RuntimeState	Scene data fully available at runtime

File	Nature of Change	Enables
compiler/runtime/state/parameter_state.rs	Add <code>display_symbol</code> , <code>description</code> , <code>animation_entity_id</code> to <code>Parameter</code>	Variable Inspector; animated parameter response
compiler/renderer/visibility.rs	Add <code>apply_focus_with_token()</code> ; add <code>highlight_token</code> to renderer <code>ObjectState</code>	Token-aware colored highlighting
compiler/renderer/backend/traits.rs	Add <code>set_highlighted_with_color()</code> and <code>set_annotation()</code> to <code>RenderBackend</code> trait	Color and label support across all backends
compiler/renderer/backend/three_js.rs	Implement color highlight and annotation via JS interop	Color-matched highlights; floating text labels in canvas
compiler/runtime/engine.rs	Add <code>step_emitter</code> injection; update <code>state.current_highlight_token</code> after evaluation; load schedule/annotations on init	Engine wired to step trace; state always reflects active step